EE292D: Machine learning on embedded systems

https://ee292d.github.io/

LECTURE 4 - APRIL 27, 2020

Overview

Discussion Topics

Model Quantization

Fixed-point, N-nary, FP16, and BFloat

Lab

Model Quantization

A personal history of quantization

The first model I worked with was AlexNet. It had 61 million parameters. Stored on disk, each weight parameter took 4 bytes. This meant a 244 MB file.

I had code running on an iPhone, but the size of the file meant I couldn't run it, since I only had around 100 MB of RAM to use inside an app!

Inspecting the weight values, it became obvious that each layers' parameters were within limited ranges. They tended to be much less than 100, often between something like roughly -10 to +10.

I was looking for a compression scheme, and the simplest approach seemed like saving the minimum and maximum real numbers for each layers' weights

Example

To understand this, let's look at a real model, using a Colab notebook:

https://drive.google.com/file/d/1Pn-lNOty-P1fX6G66oa6wvsEn46Xyx8Y

This loads the Inception v3 model from 2015 as a TF Lite model, and inspects the weights of the first convolution layer.

It will take about ten minutes to complete if you run it yourself, but for now let's look at the stored results.

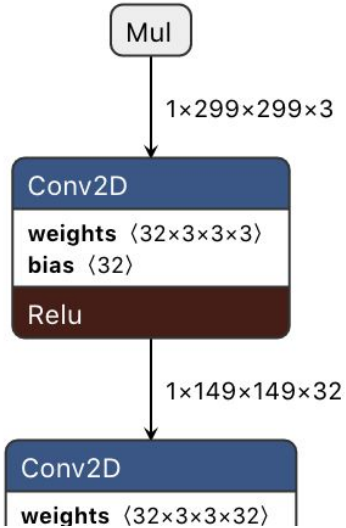
Example

You can download the model from https://storage.googleapis.com/download.tensorflow.org/models/tflite/inception_v3_2015_2017_11_10.zip

If you download the archive and then load the .tflite file into https://lutzroeder.github.io/netron/ you'll be able to visualize the model.

Example

We'll look at the weights on the first convolution layer, which has 32 filters, each 3x3 with 3 input channels (RGB). This means there are 864 elements in the array.



Example

The range of the float values is from -5.4 to 4.5:

```
[ ]  params.min()
```

```
-5.4024096
```

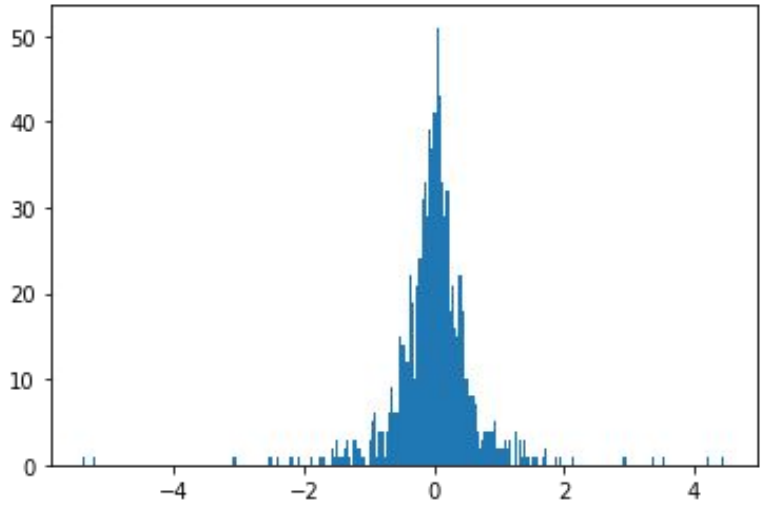
```
[ ]  params.max()
```

```
4.4646473
```

Example

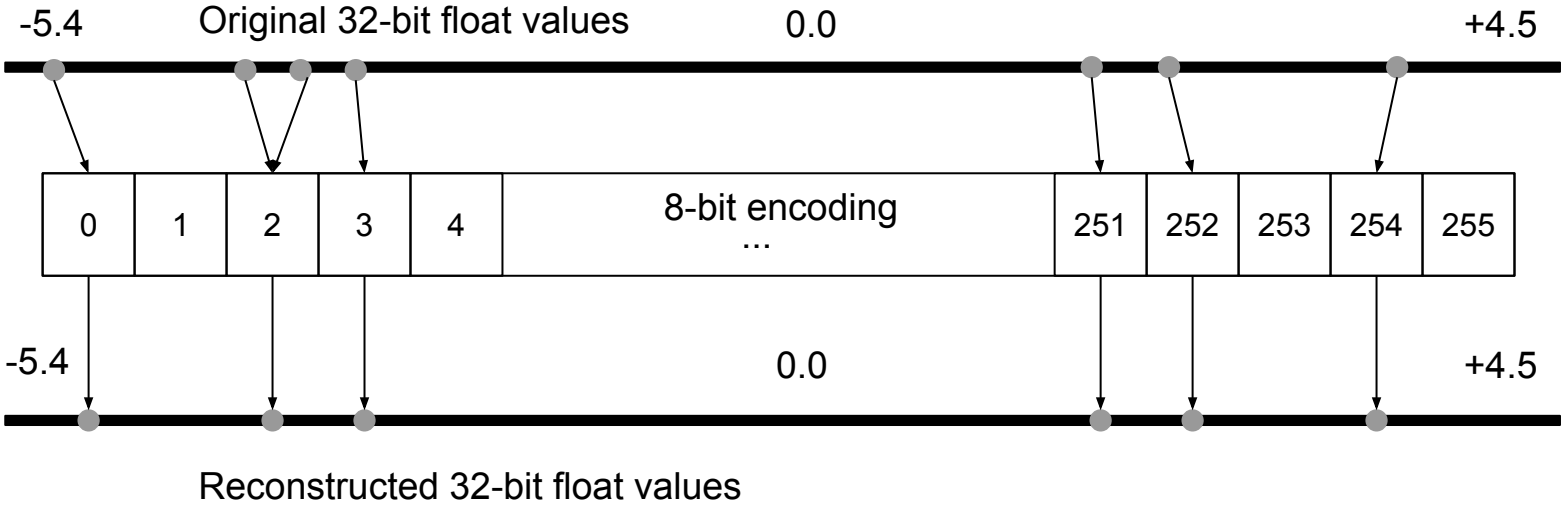
The histogram plot shows some interesting properties:

- There's a tight, mostly-symmetrical distribution around zero.
- There are very few values above or below 2.0.
- A lot of the histogram slots are empty.



Quantizing a weight layer

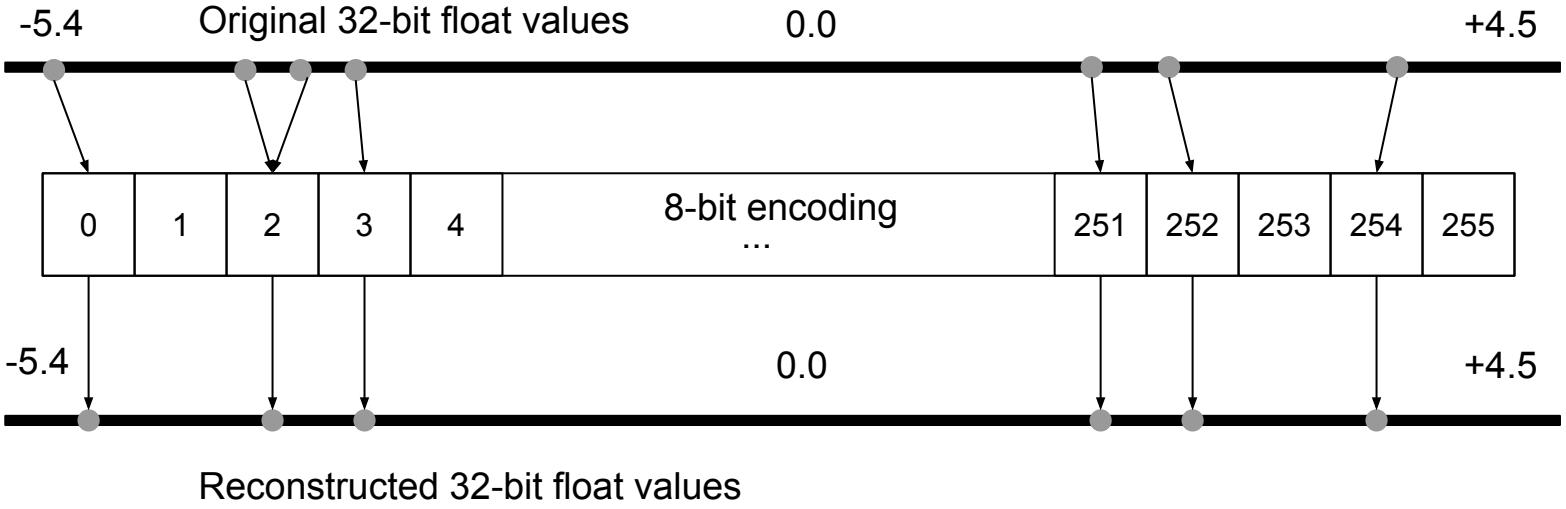
Looking at properties like these made me think that compressing the 32-bit float values into 8-bit codes by linearly encoding between the maximum and minimum values might work without sacrificing too much accuracy.



Quantizing a weight layer

Psuedo-code is:

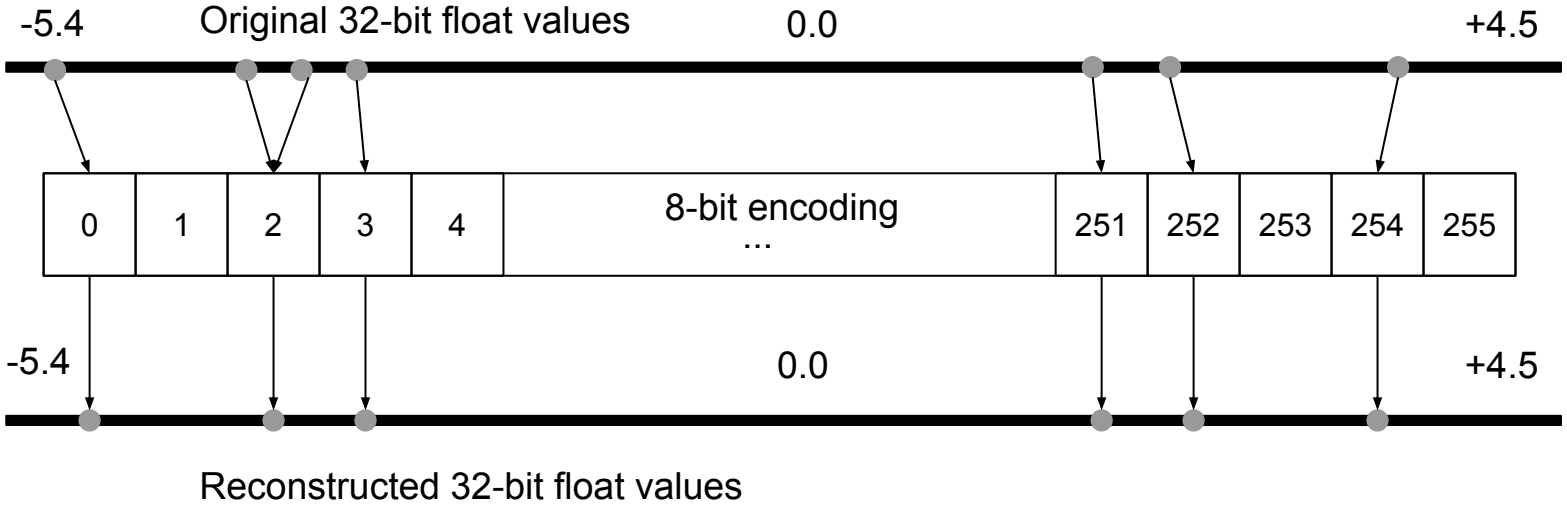
```
quantized_code = uint8(((original - min) / (max - min)) * 255)
reconstructed = ((quantized_code / 255.0) * (max - min)) + min
```



Things to notice

This is just a file compression method, we reconstruct floats for calculations.

There's a rounding error when decompressing, max of +/-(range/256)/2.



Results

By loading and reconstructing the weights in the layer implementation, I was able to add support with minimal code changes. The accuracy and latency were mostly unaffected! It reduced the size of AlexNet from 240MB to 62MB, which allowed me to deploy it on an iPhone and the web.

I wasn't the first or the only person to discover this approach, but there wasn't anything in the ML research literature early on, since it was seen as an engineering detail. Was used by TPU v1 for example, designed before I joined Google in 2014.

That started to change with Deep Learning with Limited Numerical Precision in 2015, followed by Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference in 2017.

Why is accuracy unaffected?

Most numerical algorithms suffer when precision is reduced, why not neural networks?

My intuition is that networks are trained to be robust to noisy inputs in domains like images, audio, NLP. The rounding errors introduced by 8-bit quantization are smaller than the average noise that propagates through the model.

I haven't been able to demonstrate this through experiments though, so take this explanation with a pinch of salt.

Implications

Neural networks' robustness to precision errors opens up a lot of engineering questions:

- Can we perform calculations in lower precision too?
- How about training as well as inference?
- What about lower bit depths than 8-bit?
- Does this fit with existing hardware?
- What about new hardware designs?
- Weight distribution is non-uniform, is a linear mapping the best way?

Practical notes

Rounding is the work of the devil:

- Models are robust to random errors, but a bias (where errors don't sum to zero on average) will spoil accuracy quickly.
- The exact rounding method you use for coding and reconstructing is crucial to avoid a bias. It's easy to have errors creep in.
- Exact encoding of 0.0 is crucial, since it's the most common weight value and shows up with padding too. Any encoding error for zero has an oversize effect on the result because it's so frequent.

Extreme outliers matter for weights:

- You might look at the distribution and think we could avoid a lot of empty wasted coding slots by picking the 95/5 percentiles.
- It turns out the extreme values are very important for accuracy.

Rounding to floats and then zipping works!

Advanced uses for quantization

Model compression:

- Basics already covered, will show how to do it in TensorFlow Lite.

Inference calculations:

- Lots of work in this area, very useful for embedded, will explain more.

Training calculations:

- Not the focus of this course, but will discuss briefly.

The Big Picture

Or Why Quantization Matters

Neural networks are radically different workloads

Traditional numerical computing focuses on analytical tasks that demand high levels of precision to complete. Any numerical errors compound through subsequent steps.

Neural networks are exercises in pattern matching on very noisy inputs. To work at all they have to build in redundancy and robustness to errors.

Neural workload properties

Analytical Calculations	Neural Networks
Small numerical errors compound	Small numerical errors cancel out
Many branches and memory accesses are data-dependent, and not known until shortly before use	Calculations and data accesses are data-independent, and can be precisely predicted for millions of cycles ahead
The number of data accesses is equal to or greater than the number of arithmetic operations (high memory intensity)	The number of data accesses is much lower than the number of arithmetic operations (high arithmetic intensity)
Programs are made up of many small functions interspersed with business logic supplied by the developer in a general purpose language	Programs are made up of a limited number of high level “layers” that express large math operations

Hardware response

Neural Networks	Hardware
Small numerical errors cancel out	Only support eight-bit and lower arithmetic
Calculations and data accesses are data-independent, and can be precisely predicted for millions of cycles ahead	Scrap predictive caches (or never add them in the first place), add heavily-pipelined hardware for arithmetic
The number of data accesses is much lower than the number of arithmetic operations (high arithmetic intensity)	Pump up the arithmetic capabilities (there's a lot of headroom)
Programs are made up of a limited number of high level "layers" that express large math operations	Support just a few NN frameworks, instead of general-purpose languages and libraries with larger footprints

Cambrian explosion of software and hardware

An anonymous hardware architect told me “Things were getting very predictable until neural networks came along, they made life exciting again”!

We have an increasing number of applications with radically different requirements of the systems they run on, both software and hardware. This is forcing the whole industry to rethink their solutions.

There have been some unexpected winners. For example, DSPs are designed to do SIMD eight-bit multiply-adds very fast at low power, which is exactly what we need.

Quantization is the most visible example of these changed needs.

What you're learning

We're focused on embedded applications, but engineering systems that use neural networks requires a lot of knowledge and skills that aren't widely known.

I'm excited about this course because it's my chance to share some of what I've learned (much of it from colleagues)!

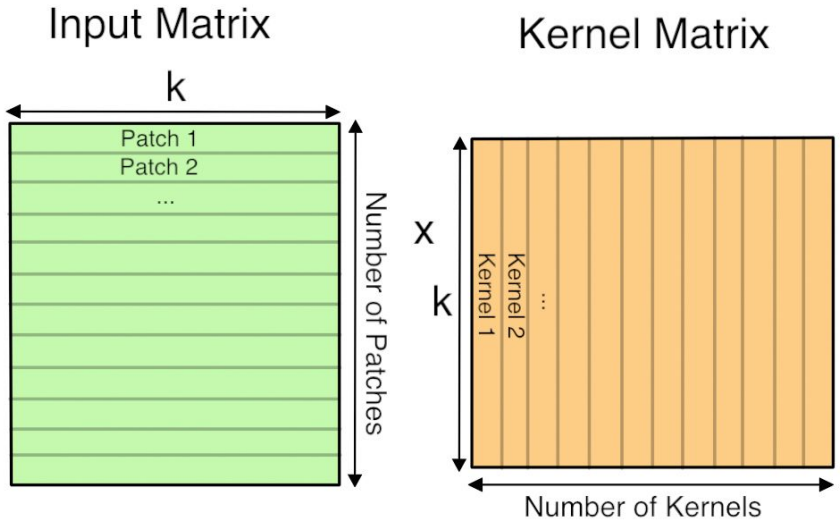
Knowing how neural networks behave will be crucial engineering knowledge as their deployment grows.

Quantized Inference Calculations

Quantized Weight Compression

The initial example showed how to shrink the file size, but the underlying arithmetic was still done in floating point.

An obvious extension is to calculate using eight bits as well. Refresher on what the math involved is:



Quantized Weight Compression

Or in pseudo-code:

input : Input feature map, I, convolution parameters W
output: Output feature map, O

```
for k ← 0 to C_o do ;                    // Output Channels
 for y ← 0 to H do ;                        // H: Height
  for x ← 0 to W do ;                        // W: Width
   O[k][y][x] = 0;
   for c ← 0 to C_i do ;                   // Input Channels
    for j ← 0 to K_h do ;                    // Filter Size
     for i ← 0 to K_w do
      O[k][y][x] += I[c][y+j][x+i] * W[k][c][j][i]
```

Quantized Weight Compression

The inner loop is effectively a dot product of a portion of the activation input and the weights.

`output = (input0 * weight0) + … + (inputn * weightn)`

```
O[k][y][x] = 0;
for c ← 0 to C_i do ;                    // Input Channels
 for j ← 0 to K_h do ;                     // Filter Size
  for i ← 0 to K_w do
   O[k][y][x] += I[c][y+j][x+i] * W[k][c][j][i]
```

Quantized Weight Compression

With weight compression we decompress each W value from eight bits into a float before multiplying it with the input value:

```
output = … inputn * decompress(q_weightn)
```

Where:

```
decompress(quantized_code) {
 return float((quantized_code / 255.0) * (max - min)) + min;
}
```

Quantized Inference Calculations

Imagine that we artificially reduce the precision of every input to the dot product, so that they're no longer using the full range of a 32-bit float:

```
output = … quantize(inputn, step) * quantize(weightn, step)
```

Where:

```
quantize(x, step) {
 return round(x*step) / step;
}
```

For example, quantize(3.14, 1.0) = 3.0 (rounding to nearest whole number), and quantize(3.14, 0.1) = 3.1 (rounding to nearest 1/10).

Why?

Doing all calculations in eight-bit integers offers some compelling advantages:

- Faster arithmetic. You need a lot fewer gates to implement an eight-bit integer multiply-add than a 32-bit floating point operation.
- Lower memory demands. We're only accessing eight bits instead of thirty-two, which reduces the load on the memory system by 75%.
- Reduced requirements. Many low-end microcontrollers and DSPs lack floating-point hardware, so avoiding floats increases portability.

Quantized Inference Calculations

What happens to our model accuracy with this precision reduction? Generally, if we pick a small enough step for the input and weights, the accuracy survives with very little degradation!

```
output = … quantize(inputn, step) * quantize(weightn, step)
```

You may notice that our quantize function for weights is very similar to our decompression code. We know what the min/max range for the trained weights is, so we can figure out a ‘step’ value that allows us to ensure they fit within the -128 to +127 range of a signed eight-bit integer.

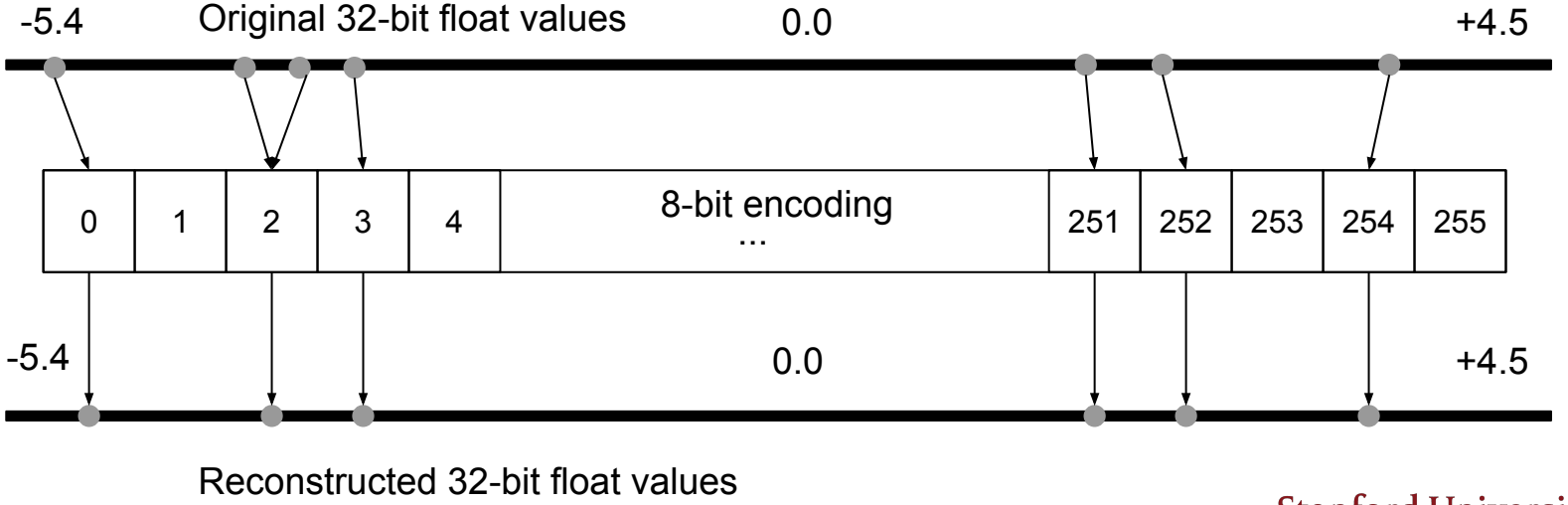
Signed eight bit?

You may have noticed a little bit of sleight of hand. The eight-bit weight compression process used 0 to 255 as codes, but I'm now talking about signed eight bit types with a range of -128 to 127 for calculations.

This is because we need to perform math with our weights, rather than just convert them into a form that's then fed into arithmetic.

Asymmetric versus symmetric

The weight compression example we used earlier had values from -5.4 to +4.5. This is no problem when we're converting to and from a compression code, because we subtract the min value to get a signed float. This is known as an asymmetric representation, because it's not centered on zero.



Asymmetric versus symmetric

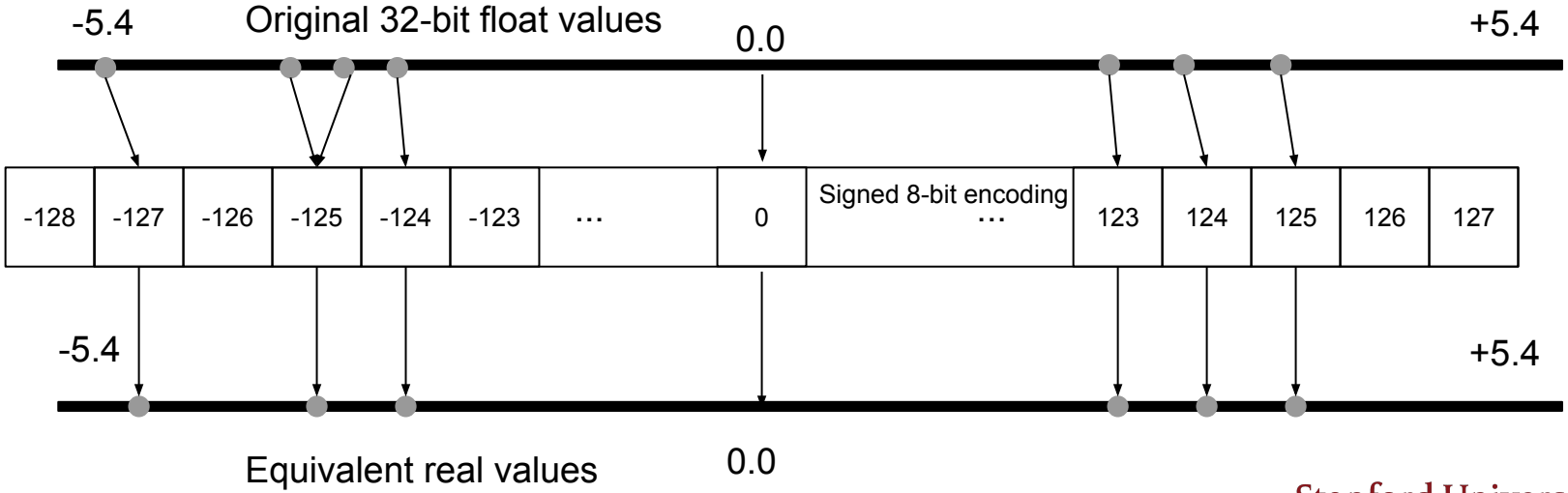
If we want to store weights as signed eight-bit integers and treat them as numbers, they need to retain some of the same properties of the real numbers they represent:

- encoded(0.0) * x should always equal zero
- encoded(-x) * y should always flip the sign of y
- There should be a linear relationship between encoded(x) and x

The reason we need these is that we want to replace the float values in the dot product with eight-bit numbers, and these are some of the attributes we need.

Asymmetric versus symmetric

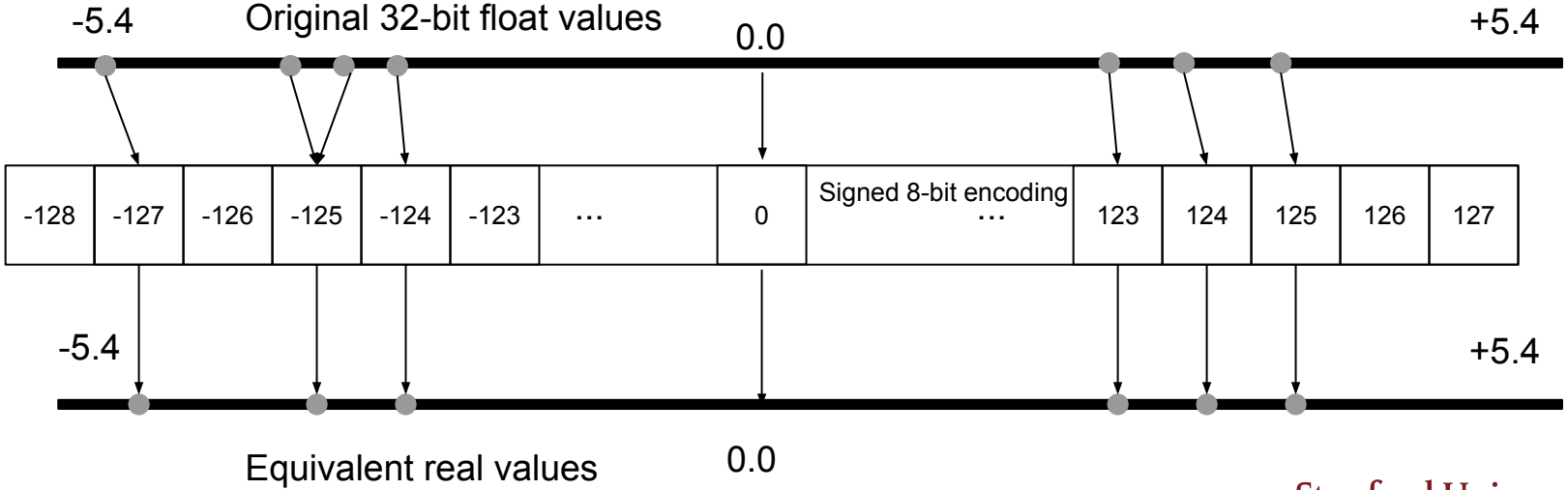
To get an encoding we can use in calculations, we need to make some changes. The range has to be the +/- the absolute maximum of the min/max range of the values. For our example, this means -5.4 to +5.4:



Asymmetric versus symmetric

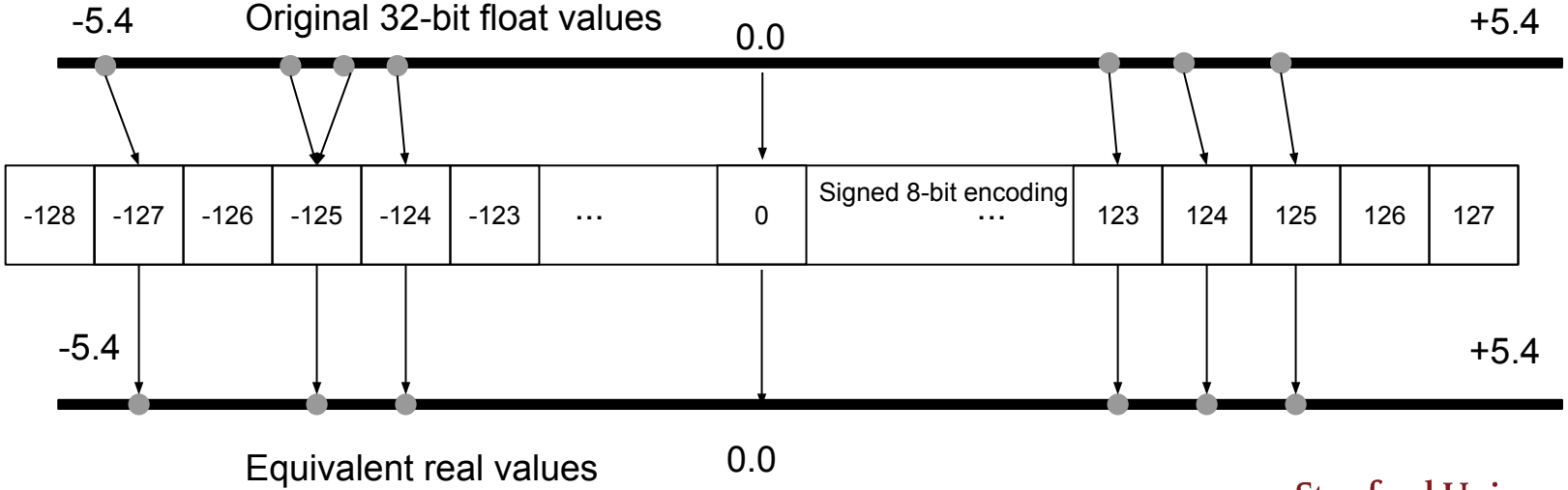
This symmetric representation has some interesting properties:

- 0.0 is always represented exactly as a zero int. This is important because zero occurs much more frequently than other numbers in our calculations, so any error there will compound and hurt accuracy.



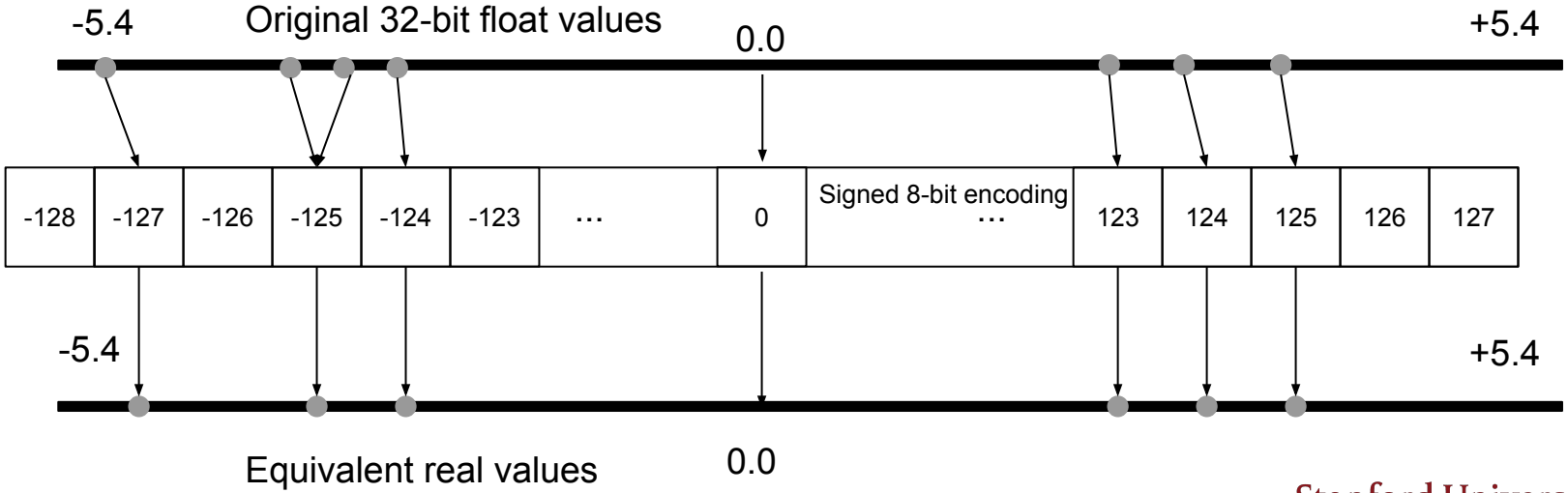
Exact zero

Requiring 0.0 to exactly map to a zero integer is awkward with a signed eight bit integer because the range is actually -128 to -127. This means we have to leave -128 unused, and map to -127 to +127, leaving us only 255 code points. Rounding here is easy to get wrong in subtle ways!



Unused range

Compared to the asymmetric representation, this wastes some of the code points on values that aren't used (from +4.5 to +5.4 in this example). This means the values are stored with slightly less precision.



Quantized Inference Calculations

Now we've come up with an eight-bit numerical representation of our values, let's use it in the dot product at the heart of our algorithm:

```
output_int32 = … input_int8_n * weight_int8_n
```

Here, all the dot product arguments are in the symmetric eight-bit integer representation we've covered.

Each multiply gives a 16-bit signed result, and they're accumulated into a signed 32-bit result.

What about the input activation quantization?

So far I've concentrated on quantizing the weights, but you might notice that we have the 'input_int8_n' term in our dot product now too.

The theory of quantizing the input activation buffer is the same as for the weights, but with one crucial difference.

We don't know what the minimum and maximum values of the input activation layer ahead of time! We need this to do the scaling to eight bits.

Figuring out the input activation range

The problem is that we can't tell from inspection what the likely minimum and maximum values for any activation layer are. They depend on both the weight and input values for the layer that generated it.

There have been a lot of attempts to solve this, each with tradeoffs.

Analyze the largest possible ranges

We probably know the range for the networks original input (for example -1.0 to 1.0 for a normalized image) so we can analytically calculate the maximum possible ranges for subsequent layers.

The problem is that these are wildly larger than the actual distributions we observe in practice, which means the encoding wastes most of its range on never-used values, and the precision for the used range is very low, resulting in poor accuracy.

Runtime min/max calculations

As we're executing each layer we can observe what the min/max values are for its output, and use that as our encoding range.

This is transparent to the user and makes optimal use of the encoding space.

It does require runtime inspection of the activation buffers and adjustment of the quantization parameters during inference, which is not possible on some hardware.

Training logging

It's possible to keep track of statistics on the min/max range of activation layers during training.

This works well, but requires that the deployment team is able to modify and run the training script, which is often not possible for organizational reasons.

Having this kind of access does allow in-training quantization too though, which is discussed later.

Training restrictions

Activation functions have the nice property of restricting the ranges of their outputs. For example, Relu ensures that its outputs are always positive, and others like TanH are between fixed limits.

By picking only activation functions that ensure a range (such as Relu6 that saturates out at 6.0 as well as a minimum of 0.0) it's possible to know what the possible activation ranges are, purely from the architecture.

This can have a negative impact on the overall accuracy of the model though. Anecdotally, I've seen the limited range seem to 'inflate' the weights that feed into it for example, though I've not experimented enough to tell more.

Post-training calibration

Once we have a float trained inference model, it's possible to take some example inputs and run them through to observe the activation ranges.

This works well, but setting up the calibration input data set and feeding it in is painful.

This is what we recommend for TensorFlow Lite.

Activation outliers

Unlike weights, I've found outliers to be less important for activation inputs, so using the 95/5 percentiles as the limits seems to work well.

Make sure that your encoding function saturates beyond min/max ranges though, or you'll end up with nasty wraparound errors.

Accumulated result

I mentioned that ‘output_int32’ is the accumulated result of the dot product, but let’s look at that a bit more closely.

```
output_int32 = … input_int8_n * weight_int8_n
```

We know each multiply produces a 16-bit signed result, and there are typically a few hundred in each dot product. This tells us that the sum of a few hundred 16-bit ints won’t take more than 25 or 26 bits total. In practice many of the multiply are near zero, so we often see only 20-ish bits used.

Modern CPUs only support 32-bit registers though, so we can’t take much advantage of that.

16-bit accumulation

There has been some experimentation with accumulating into 16-bit accumulators for two or more multiply results. If the hardware supports faster 8x8 multiply-adds into 16-bit accumulators this is attractive, as long as they saturate.

Almost none of the results will be near the maximum, so it's generally safe to accumulate quite a few into a limited range type, though your mileage may vary.

They will eventually need to be combined into a higher bit-depth register at the end.

Scaling the accumulated result

At the end of our dot product, we have a signed 32-bit accumulated total.

The following operation needs a signed eight-bit number. How do we get there?

We know that the real values the accumulator represents are equal to (input_step * weight_step).

For example, if the weights are -10.0 to 10.0, and the activations are -5.0 to 5.0, then the weight step is 10/127 and the input step is 5/127. That means each unit in the accumulator represents about (10/127)*(5/127) = 0.0031.

So an accumulated value of 97 would represent 97*0.0031 = 0.3007

Scaling the accumulated result

From whatever activation range method we used, we know the expected eight-bit range of the output of our layer. We also know how to convert from accumulated units to real values. Using that knowledge we can figure out how to do the scaling.

For example, using the previous values if we know the output should be between -300.0 and +300.0, we want -127 to represent -300.0 and +127 to be +300.0.

We can calculate that 300.0 is (300.0/0.3007) = 997.67 in the accumulated representation. If we scale all the values by (127/997.67)=0.1273 and then clamp to -127 to +127 we will get the correct representation in the eight-bit output activation layer.

Scaling the accumulated result

The calculations were shown with real values, but we need to do them using only integer arithmetic for both efficiency reasons, and in case we're on a device with no floating point hardware.

We usually do some fixed-point math instead, something like:

```
result = (accumulated * scaling) >> X
```

The tough part is that it's easy to overflow beyond 32-bits, depending on the values of 'scaling' and 'X', or alternatively use too little precision. Getting the rounding wrong is easy too, so in our code this is actually a surprising performance bottleneck!

Per-filter scaling

Also known as per-channel or per-axis scaling.

So far we've been talking about ranges across the entire set of weights fed into a layer. Convolution weights have the intriguing property that each filter has its own natural range though, which is a subset of the collection they're in. For example a few filters might have values ranging from -10.0 to 10.0, but other might be smaller, running -1.0 to 1.0. The overall collection of filters would adopt the maximum range, so those smaller filters would only use a small part of the coding, so missing out on precision.

It turns out we can scale each filter's weights to fit their range, and then cheaply account for the difference during the accumulation scaling.

Per-filter scaling

The dot product we're doing looks like this:

```
output_int32 = … input_int8_n * weight_int8_n
```

Each output channel is only using weights from a single convolution kernel. We're scaling the 'output_int32' by a single fixed multiplier to convert it into an int8 for the next layer anyway. If we normalize all the weights for a single kernel into the range for that kernel, and then have an inverse multiplier added to the accumulator scaling factor, we can get the increased precision almost for free!

The only change is that we have a different fixed scaling factor for each output channel, so the scaling becomes multiplying by a vector instead of a scalar.

Asymmetry strikes back

Another optimization is noticing that we can actually calculate the offsets needed for asymmetrical ranges of activations ahead of time, so we can get the precision benefits of an increased range for free:

https://www.tensorflow.org/lite/performance/quantization_spec#symmetric_vs_asymmetric

A is a $m \times n$ matrix of quantized activations.
B is a $n \times p$ matrix of quantized weights.
Consider multiplying the jth row of A, a_j by the kth column of B, b_k, both of length n. The quantized integer values and zero-points values are q_a, z_a and q_b, z_b respectively.

$$a_j \cdot b_k = \sum_{i=0}^{n} a_j^{(i)} b_k^{(i)} = \sum_{i=0}^{n} (q_a^{(i)} - z_a)(q_b^{(i)} - z_b) = \sum_{i=0}^{n} q_a^{(i)} q_b^{(i)} - \sum_{i=0}^{n} q_a^{(i)} z_b - \sum_{i=0}^{n} q_b^{(i)} z_a + \sum_{i=0}^{n} z_a z_b$$

The $\sum_{i=0}^{n} q_a^{(i)} q_b^{(i)}$ term is unavoidable since it's performing the dot product of the input value and the weight value.

The

$\sum_{i=0}^{n} q_b^{(i)} z_a$
and
$\sum_{i=0}^{n} z_a z_b$
terms are made up of constants that remain the same per inference invocation, and thus can be pre-calculated.

The $\sum_{i=0}^{n} q_a^{(i)} z_b$ term needs to be computed every inference since the activation changes every inference. By enforcing weights to be symmetric we can remove the cost of this term.

TensorFlow Lite support

As you may be able to tell, it's been a winding road getting to this point!

You can see the current state of the art with post-training quantization in TensorFlow Lite here:

https://www.tensorflow.org/lite/performance/quantization_spec

There's other great work happening elsewhere too, for example:

https://ai.facebook.com/blog/training-with-quantization-noise-for-extreme-model-compression/

What about RNNs?

I've concentrated on convolutional networks, and recurrent networks have different properties. They involve a lot of small incremental changes to stored values, which means they have different properties to CNNs.

They have been successfully quantized to eight bits for production use cases, but there hasn't been much work published on this process. I'm hoping this will change soon.

What about fixed-point?

There's a long tradition of using fixed-point arithmetic in computing. In terms of our representation, this is the equivalent of always using powers of two as our real ranges, so that -10.0 to 10.0 would become -16.0 to 16.0, and then we could represent it as SDDDDFFF, where the top bit is the sign, the next four bits are the units, and the final three bits represent the fraction.

The trouble is that we're constantly trying to preserve as much accuracy as possible. A fixed-point representation wastes some of the range, and hurts overall model accuracy, so we haven't been able to standardize on this.

It is tempting from an implementation efficiency perspective though, maybe we can get there eventually?

What about less than eight bits?

There has been some great work on lower bit depths. We're focused on eight bits as the sweet spot right now because:

- Only a few hardware devices (other than FPGAs) support bit depths other than eight or one-bit efficiently.
- One or two-bit networks haven't been able to achieve the same accuracy as eight-bit models, even with many more operations (as far as I'm aware from published work, xnor.ai seemed to have promising results, I hope this changes).
- It's hard to implement the xnor/pop-count sequence efficiently on existing devices. OctoML are making progress though.

Quantized Training

Using reduced precision for training

So far I've talked about inference, how can we use the same insights for training? It's a lot harder:

- We don't have fixed weight or inference ranges, they're constantly changing.
- The training process involves lots of extremely small increments to values
- Desktop GPUs tend to be used for training, and they've traditionally been very float-focused

I've also spent most of my time on inference, so apologies if I miss prior work in this round-up.

Using eight bits for weights and activations

Most increments are much less than the lowest-significant bit, so weights never change.

I've tried stochastic updating, so even small changes have a chance of updating. I've never got good results.

16-bit training

There's a much richer tradition of using 16-bit representations during training. I've seen 16-bit fixed-point used, but early in training the ranges can be so large it's hard to pick the right scale factors. Most work has focused on floating-point representations.

FP16

FP16 is the IEEE standard for 16-bit floating point, originally coming from the graphics community for representing dynamic-range images. As such, it's been well-supported by GPUs.

It has one bit for the sign, 5 bits for the exponent, and ten bits for the fraction.

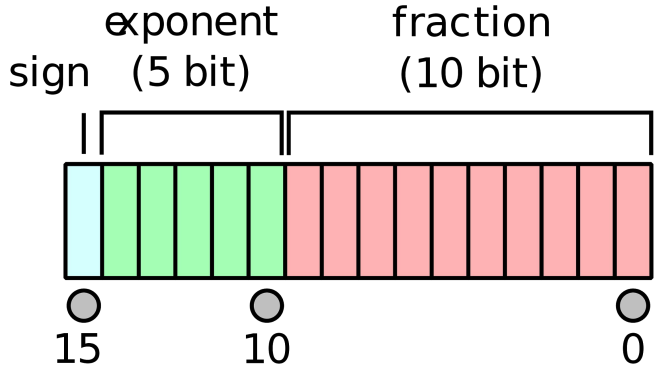
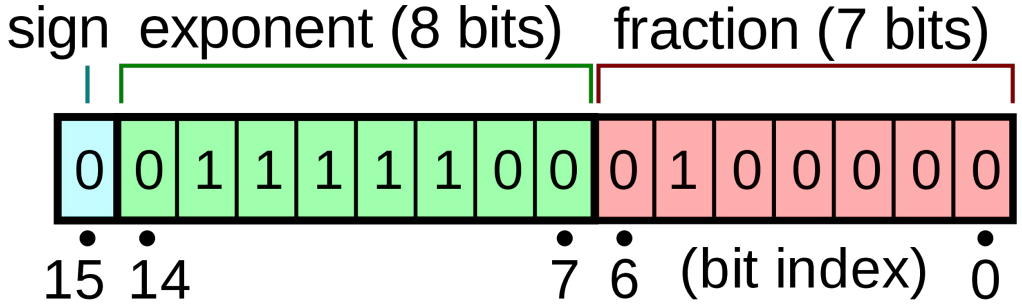


Image via https://en.wikipedia.org/wiki/Half-precision_floating-point_format

BFloat16

BFloat16 is an alternate format that uses 8 bits for the exponent and 7 bits for the fraction:



It's been found to work better with training because the exponent gives a larger range, and the fraction is less important for good results. It's used by Cloud TPUs and other accelerators.

Conversion is also easy, because it's just the top 16 bits of an IEEE float32.

Quantization-aware training

Even if you're training in 32-bit float, if you know you're targeting quantized inference it can be helpful to use that knowledge during training:

- You can record accurate quantization ranges for activations.
- By injecting the sort of rounding errors that occur during quantization, you can make the network more resilient to them and increase the overall accuracy.

TensorFlow has support for this kind of quantization-aware training:

https://blog.tensorflow.org/2020/04/quantization-aware-training-with-tensorflow-model-optimization-toolkit.html

Questions?

I'm @petewarden on Twitter, there's a YouTube playlist of screencasts on embedded ML, and you can get the first six chapters of the TinyML book free at tinymlbook.com

